

Academy Backend/Frontend/QE Virtual MTY

Proyecto: 02-Spring-Security

Desarrollador: Sergio Servando Prado Lozano

Encargado: Miguel Ángel Rugerio Flores

Lugar: San Pedro de las Colonias Coahuila

Fecha: 29/08/2026

Concepto 2.1: HTTP Basic y BCrypt

Si publicamos una API sin seguridad, es como dejar una base de datos abierta en internet cualquiera con la URL podría borrar nuestra información. El mecanismo de HTTP Basic resuelve la Autenticación demostrando quién eres exigiendo que envíes tu usuario y contraseña en la cabecera de cada petición HTTP. Además, resuelve la Autorización qué puedes hacer validando si tienes el rol necesario EMPLOYEE, MANAGER o ADMIN.

Para demostrar esto, se hizo una arquitectura completa en `src/main/java`. Lo más fascinante de Spring Security es la Cadena de Filtros: la seguridad intercepta la petición antes de que toque nuestra lógica de negocio. Así funciona la estructura:

El Dominio y Acceso a Datos (entity, dao y service):

- En `entity/Employee.java` defino mi modelo de base de datos (el empleado con nombre, apellido y correo).
- En `dao/EmployeeRepository.java` uso Spring Data JPA para no tener que escribir las consultas SQL a mano.
- En `service/EmployeeServiceImpl.java` manejo la lógica de negocio (buscar, guardar, eliminar).

Ninguna de estas clases sabe que existe la seguridad. Están siempre enfocadas en los datos.

El Controlador REST (`rest/EmployeeRestController.java`):

- Aquí se muestran los endpoints (`@GetMapping`, `@PostMapping`, `@DeleteMapping`).

- En el código de deleteEmployee(), no hay ni un solo if (usuario.esAdmin()). El controlador asume que si la petición llegó hasta ahí, es porque Spring Security ya revisó las credenciales y le dio permiso de pasar.

El Escudo de Seguridad (security/SecurityConfig.java):

Aquí es donde sucede toda la magia. Configuré dos Beans principales:

- userDetailsService: Resuelve la pregunta ¿De dónde salen los usuarios? Le indica a Spring que se conecte a mi base de datos con el DataSource y se ejecutan dos consultas SQL nativas sobre las tablas members y roles para buscar al usuario que intenta entrar.
- filterChain: Resuelve la pregunta ¿Qué puedes hacer? Aquí se define la matriz de roles. Por ejemplo, la línea .requestMatchers(HttpMethod.DELETE, "/api/employees/**").hasRole("ADMIN") es la que bloquea los borrados a menos que seas el jefe. También se activa .httpBasic() para que exija credenciales en cada petición, y se configura la sesión como STATELESS (sin estado) para que el servidor no guarde memoria de quién entró antes.

En la base de datos aquí se muestra como es que se estructura.

Esta es mi tabla employee:

				id	first_name	last_name	email
☐	 Editar	 Copiar	 Borrar	1	Prriqui	Prrio	prrio@gmail.com
☐	 Editar	 Copiar	 Borrar	2	Prrio	Paps	paps@gmail.com
☐	 Editar	 Copiar	 Borrar	3	Ada	Lovelace	ada@gmail.com

Esta es mi tabla de members y con las buenas prácticas de la contraseña:

							user_id	pw	active
☐		Editar		Copiar		Borrar	john	{bcrypt}\$2y\$10\$q5C89SltU5ZKPZITspXrZO0cm7njHEeRF7d...	1
☐		Editar		Copiar		Borrar	mary	{bcrypt}\$2y\$10\$y0UvRlnLKIOh7nBfH8sNvuXUIVhwOMYaz1...	1
☐		Editar		Copiar		Borrar	susan	{bcrypt}\$2y\$10\$6eOesXI7A1E7kaE7UYulPu4h5o5r6Yqd/F/...	1

Y aquí en la siguiente tabla se muestran los roles:

							user_id	role
☐		Editar		Copiar		Borrar	john	ROLE_EMPLOYEE
☐		Editar		Copiar		Borrar	mary	ROLE_EMPLOYEE
☐		Editar		Copiar		Borrar	mary	ROLE_MANAGER
☐		Editar		Copiar		Borrar	susan	ROLE_ADMIN
☐		Editar		Copiar		Borrar	susan	ROLE_EMPLOYEE
☐		Editar		Copiar		Borrar	susan	ROLE_MANAGER

Atendiendo a las buenas prácticas y al temario, las contraseñas de mis usuarios nunca se guardan en texto plano (como test123). En la base de datos MySQL, se guardan utilizando BCrypt (identificables por el prefijo {bcrypt}). BCrypt es un algoritmo de una sola vía que usa un salt (un texto aleatorio por cada usuario) y tiene un "coste" de procesamiento lento a propósito. Si un atacante se roba la base de datos, le sería matemáticamente imposible revertir esos hashes para descubrir las contraseñas originales. Spring compara la contraseña que envía el usuario contra este hash de forma automática.

Aquí se mostrarán las ejecuciones en tiempo real:

```
Windows PowerShell X Windows PowerShell X + v
PS C:\Users\sergi> curl.exe -i http://localhost:8071/api/employees
HTTP/1.1 401
WWW-Authenticate: Basic realm="Realm", charset="UTF-8"
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Length: 0
Date: Fri, 28 Aug 2026 16:00:49 GMT

PS C:\Users\sergi> |
```

Este es un error porque se intenta hacer un GET sin tener los permisos necesarios.

```
Windows PowerShell X Windows PowerShell X + v
PS C:\Users\sergi> curl.exe -i -u john:test123 -X DELETE http://localhost:8071/api/employees/1
HTTP/1.1 403
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Fri, 28 Aug 2026 16:01:35 GMT

{"timestamp":"2026-08-28T16:01:34.971Z","status":403,"error":"Forbidden","path":"/api/employees/1"}
PS C:\Users\sergi> |
```

Este es un error ya que el usuario john intenta hacer una petición a la cual no tiene permiso ya que solo el puede hacer GET y pide DELETE.

```
Windows PowerShell
PS C:\Users\sergi> curl.exe -i -u susan:test123 -X DELETE http://localhost:8071/api/employees/3
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: text/plain; charset=UTF-8
Content-Length: 23
Date: Fri, 28 Aug 2026 16:04:27 GMT

Deleted employee id - 3
PS C:\Users\sergi> |
```

Esta fue una ejecución exitosa ya que Susan tiene permiso de ADMIN y puede hacer todo incluyendo eliminar.

				id	first_name	last_name	email
☐		Editar		Copiar		Borrar	1 Prriqui Prrio prrio@gmail.com
☐		Editar		Copiar		Borrar	2 Prrio Paps paps@gmail.com

Aquí está el empleado eliminado.

Concepto 2.2: JSON Web Tokens (JWT)

En HTTP Basic teníamos un problema grave: la prueba de identidad (la contraseña) viajaba en todas las peticiones. JWT (JSON Web Token) resuelve esto cambiando la contraseña por un tipo pase. El usuario manda su contraseña una sola vez, el servidor la valida y le devuelve un token firmado digitalmente que contiene sus roles y una fecha de caducidad. A partir de ahí, el usuario solo envía el token. Esto también resuelve el problema de rendimiento: validar un JWT es una simple comprobación matemática de firma en memoria, por lo que ya no necesitamos hacer consultas a la base de datos en cada petición.

Para implementar esto, agregué la dependencia `spring-boot-starter-security-oauth2-resource-server` en el `pom.xml` y se reestructuró la seguridad. Así funciona el código:

La Taquilla (`rest/AuthController.java`):

- Creé un nuevo endpoint (POST `/api/auth/login`). Aquí es el único lugar donde el usuario envía su contraseña.
- Dato clave: Mi método `login(Authentication authentication)` no lee la contraseña ni consulta la base de datos a mano. Para cuando el código llega a esta línea, Spring ya validó las credenciales. Lo único que hace mi controlador es armar el Payload (los claims como `iss` (quién lo emite), `exp` (cuándo caduca), `sub` [quién es] y los roles) y lo firma usando mi llave privada RSA (`jwtEncoder`). Al final, devuelve un JSON con el `accessToken`.

El Escudo de Seguridad (security/SecurityConfig.java):

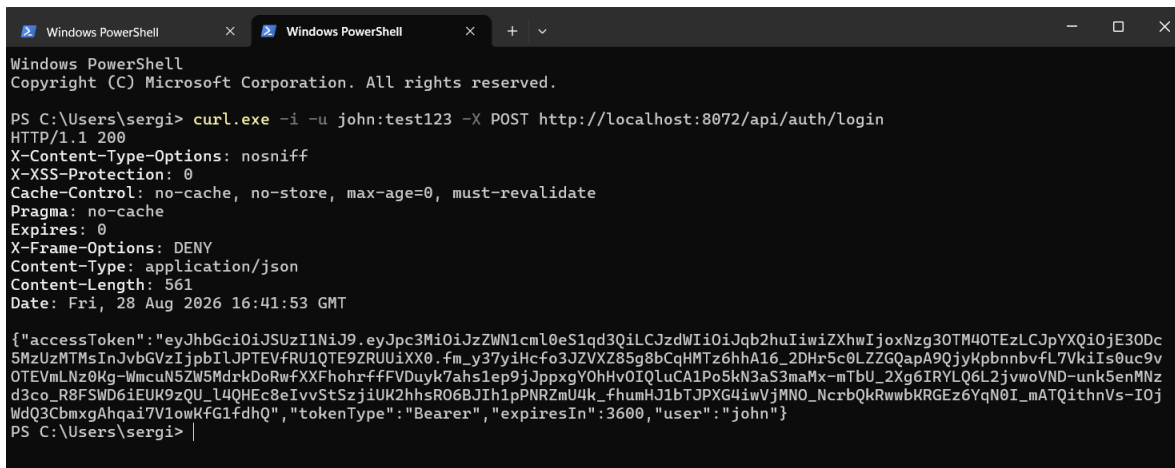
En lugar de una cadena de filtros, ahora hay dos:

- loginFilterChain (@Order(1)): Es la taquilla. Solo aplica para /api/auth/**. Funciona con HTTP Basic y va a la base de datos a comprobar credenciales.
- apiFilterChain (@Order(2)): Es el torniquete. Aplica para el resto de la API (/api/employees/**). Aquí quité el HTTP Basic y lo reemplacé por oauth2ResourceServer(oauth2 -> oauth2.jwt(...)). Esta cadena solo entiende tokens (Bearer Token).
- Las Llaves RSA: Se inyecta private.pem y public.pem. La llave privada se usa en el bean jwtEncoder para firmar (emitir) los tokens. La llave pública se usa en el bean jwtDecoder para validar que el token no haya sido alterado por un hacker.
- Traductor de Roles: Escribí el método jwtAuthenticationConverter() para evitar que Spring le ponga el molesto prefijo SCOPE_ a mis roles. Gracias a esto, mi matriz de autorización (hasRole("ADMIN")) sigue funcionando exactamente igual que en el proyecto anterior.

La Lógica de Negocio (entity, dao, service, rest/EmployeeRestController): Se quedaron intactos, ni una sola línea cambió. Los controladores siguen asumiendo que, si la petición llegó, es porque el token era válido y tenía los roles correctos.

Si nos quedamos solo con JWT sin firmar correctamente (o si usamos una llave simétrica débil), cualquiera podría decodificar el payload en base64, cambiarse el rol de EMPLOYEE a ADMIN y volver a enviarlo. Gracias a la firma con RSA256, si alguien altera un solo carácter del token, la firma matemática se rompe y mi servidor lo rechaza inmediatamente con un 401. Además, el riesgo de que roben el token se mitiga configurando una vida corta (`jwt.ttl-seconds=3600` en mi `application.properties`).

Aquí van parte de las ejecuciones:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\sergi> curl.exe -i -u john:test123 -X POST http://localhost:8072/api/auth/login
HTTP/1.1 200
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Content-Length: 561
Date: Fri, 28 Aug 2026 16:41:53 GMT

{"accessToken": "eyJhbGciOiJIUzU1NiJ9.eyJpc3MiOiJzZW50eS1qd3QiLCJzZdWIiOiJqb2huIiwiaXhwIjoxNzg3OTM4OTZLCjYXQ10jE3ODc5MTUzMTMsInR5bGVzIjpbILJPTeVfRU10TE9ZR0U1XX0.fm_y37yiHcfo3JZVXZ85g8bCqHMTz6hhA16_2Dhr5c0LZZGQapA9QjyKpbnnbvFL7VkiIs0uc9vOTEVmlNz0Kg-WmcuN5ZW5MdrkDoRwfXXFhohrffFVDuyk7ahslep9jJppxgY0hHv0IQ1uCA1Po5kN3aS3maMx-mTbU_2Xg6IRYLQ6L2jvwoVND-unK5enMNz d3co_R8FSWD6iEUK9zQU_l4QHEc8eIvvStSzjiUK2hhsR06BJIh1pPNRZmU4k_fhumHJ1bTJPG4iwVjMNO_NcrbQkRwwbKRGZ6YqN0I_mATQithnVs-I0jWdQ3CbmXgAhqai7V1owKfG1fdhQ","tokenType": "Bearer", "expiresIn": 3600, "user": "john"}
PS C:\Users\sergi>
```

Aquí una vez corrido el programa se genera la petición de login para que se le pueda generar un token al miembro el cual como en el código se menciona tiene una duración de 1 hr.

Concepto 2.3: OAuth2 y Delegación (Keycloak)

En los proyectos anteriores (Basic y JWT), mi API seguía siendo la dueña de la tabla de usuarios y gestionaba las contraseñas. El problema es que, si tengo 5 APIs distintas, tendría que mantener 5 tablas de usuarios. OAuth2 resuelve esto sacando la identidad fuera de la aplicación y delegándola a un Authorization Server (en este caso, Keycloak). La API se convierte en un Resource Server puro: ya no pregunta cuál es tu contraseña, sino quién te avala. Esto permite el Inicio de Sesión Único (SSO) y garantiza que, si mi API es hackeada, no hay contraseñas que robar porque nunca las tuvo.

La mejor forma de explicar este código es observando lo que ya no está en mi paquete `src/main/java`:

- Desapareció `AuthController`: Mi API ya no tiene endpoint de login (`/api/auth/login`).
- Desapareció `UserDetailsService`: Ya no me conecto a la base de datos para buscar usuarios. La tabla `members` ya no se usa.
- Desaparecieron las llaves `.pem` y el `JwtEncoder`: Mi API ya no emite tokens ni guarda llaves criptográficas.

Todo esto fue reemplazado por una sola línea mágica en mi archivo `application.properties`:



```
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8090/realms/academy
```

Con esta línea, al arrancar, Spring va automáticamente al puerto 8090 (donde vive Keycloak), descarga la configuración (el JWKS) y obtiene la llave pública necesaria para validar las firmas.

El traductor de Roles (SecurityConfig.java):

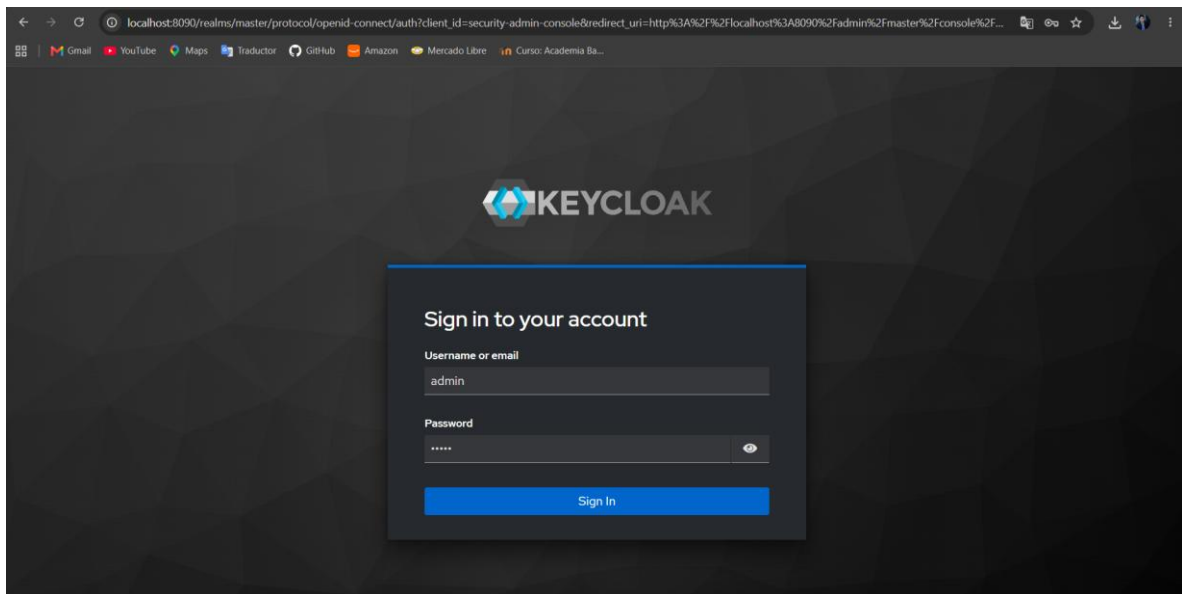
Keycloak no manda los roles en una lista plana los envía anidados dentro de un objeto JSON llamado `realm_access.roles` y sin el prefijo `ROLE_`. Se programó el método `extraerRoles(Jwt jwt)`. Este método actúa como un traductor: entra al token, navega hasta `realm_access`, extrae los roles por ejemplo `EMPLOYEE`, `ADMIN` y les concatena el string `"ROLE_"`. Gracias a esa pequeña traducción en el `SecurityConfig`, mis reglas de acceso (`.hasRole("ADMIN")`) y todos mis archivos de negocio (`Employee`, `EmployeeRepository`, `EmployeeServiceImpl` y `EmployeeRestController`) quedaron exactamente iguales en los 3 proyectos. Tres formas distintas de autenticar, pero el mismo control de autorización.

Para demostrar cómo funciona OAuth2, se necesita interactuar con dos servidores distintos (Keycloak en el puerto 8090 y mi API en el 8073):

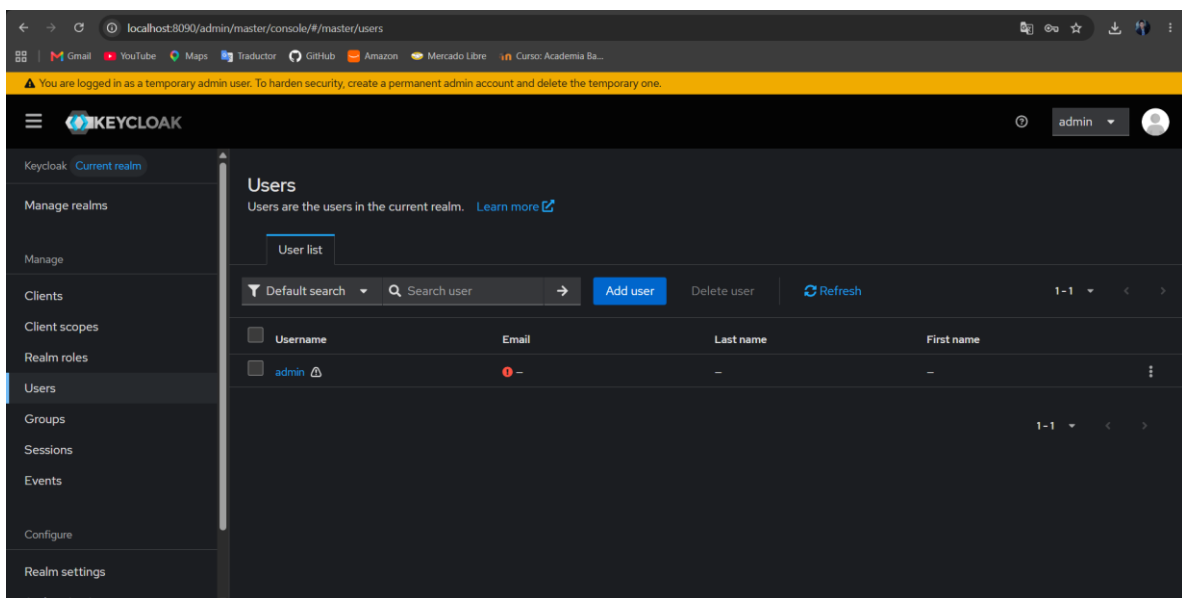
Se hace un GET a la API `http://localhost:8073/api/employees` mandando ese token como Bearer. Lo que devuelve es un 200 OK. La petición entró exitosamente. La API validó la firma contra la llave pública que descargó de Keycloak al arrancar. Y muestra los datos de la base.

[illegible]

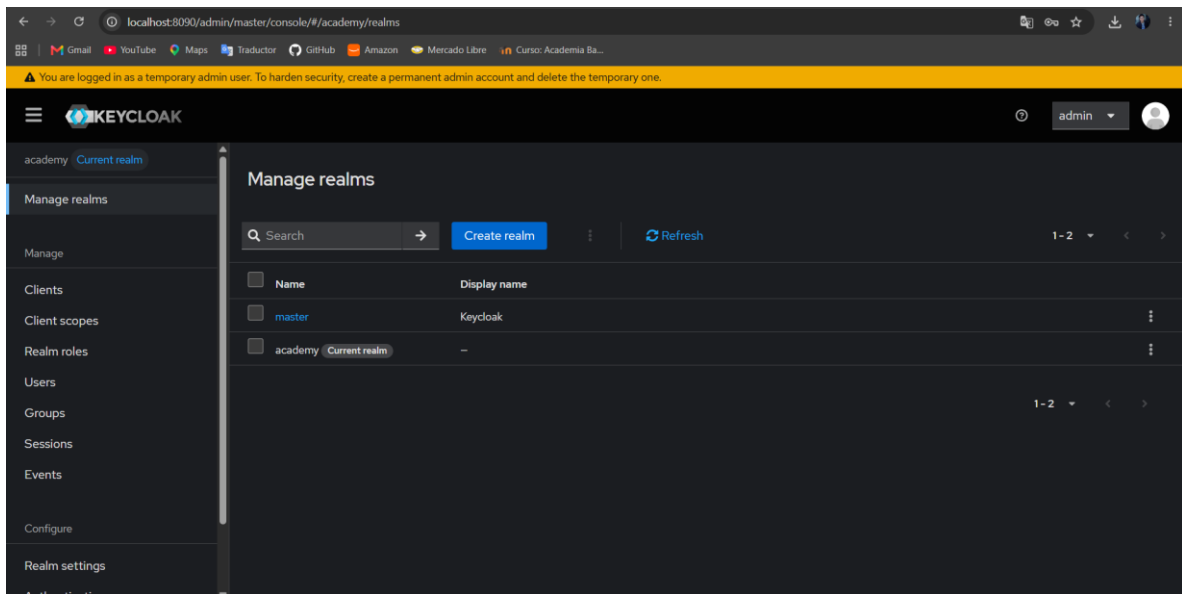
Se intenta mandarle a la API un token perfectamente válido del Proyecto 02 el JWT que firmé yo mismo con mis archivos .pem. Pero que pasa, la API lo rebota con un 401 Invalid Signature. Esto demuestra que la confianza no está en el token en sí, sino en quién lo firmó. Mi API está configurada para confiar exclusivamente en los tokens emitidos por Keycloak.



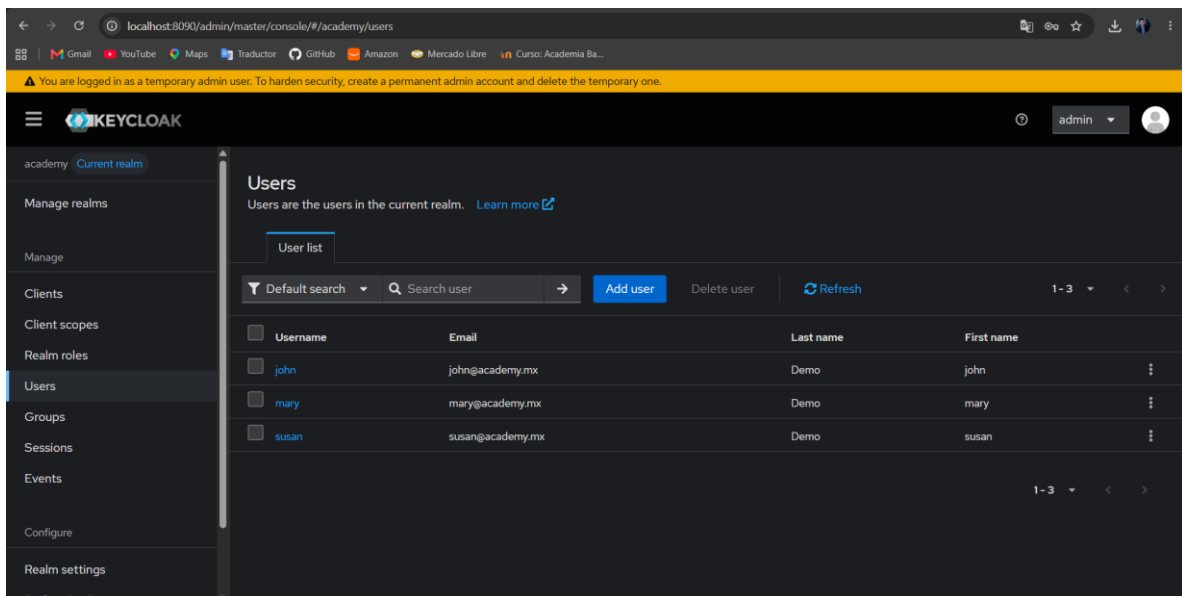
Esa es la parte visual del keylock.



En esta parte en usuarios solo aparece el admin porque estamos en el main tenemos que cambiarnos a real academy que es dónde se instancian los miembros.



Ya estamos dentro del academy y si nos vamos a users estarán ahí los miembros.



Para entender la arquitectura que se implementó, es fundamental definir a los actores del protocolo OAuth2, haciendo la analogía con un hotel algo sencillo de entender:

- Resource Owner (El dueño de los datos): Es el usuario humano, por ejemplo, john (el huésped).
- Client (El cliente): Es la aplicación que hace la petición a nombre del usuario. En mis pruebas, el cliente es CURL o Postman lo cual en algo más realista sería la parte del frontend
- Authorization Server (El emisor): En este proyecto es Keycloak (la recepción del hotel). Su única chamba es identificar a las personas.
- Resource Server (La API protegida): Es mi aplicación de Spring Boot es como la puerta de la habitación que protege los datos.

El flujo de OAuth2 divide las tareas. Cuando john quiere entrar a ver a los empleados, no me manda sus credenciales a mí. El usuario le entrega su contraseña directamente a Keycloak (Authorization Server). Keycloak comprueba la contraseña en su propia base de datos y, si es correcta, le entrega a john un Access Token (como una tarjeta-llave del hotel). Finalmente, el usuario toma ese token y me lo manda a mi API (Resource Server). Como la API solo recibe el token para validar la firma, mi código jamás recibe, lee, ni procesa la contraseña. Si mañana un hacker vulnera la API de Spring Boot, no hay ni una sola contraseña que pueda robarse porque esa información nunca cruzó por el servidor.